

Plan: GömbSoma-Stim — stimulus-driven vision architecture (content-driven hypergraph; an alternative to YOLO-style fixed-grid detection)

2026-05-14

1 Framing: the rolling-shutter vs stimulus dichotomy

YOLO, SSD, RetinaNet, and our own Phase 3-V `PatchGraphBuilder` share a *rolling-shutter* topology: a fixed grid of anchors at every position $\times$ scale, evaluated exhaustively. The grid is decided at design time, before the image arrives. Most compute is spent on background.

GömbSoma-Stim is the content-driven alternative: initial activations *vote* for where to look, top-K salient regions become anchors, hypergraph topology is built from those anchors per-image, and the existing GömbSoma Walk / Polygon / Triangle layers run on the dynamic graph. Each step is biologically motivated (Itti–Koch salience, saccadic vision, capsule routing-by-agreement).

The architectural lever GömbSoma-Stim adds over YOLO: **Cartwright–Harary σ -balance as an anchor-selection criterion**. Saliency alone is "where is the activation strong?" σ -balance is "which K regions form a structurally coherent signed substructure?" The combination filters out high-saliency-but-incoherent regions (clutter / decoys) and surfaces high-saliency-and-coherent regions (actual objects). YOLO has no analogue — it lacks signed primitives.

2 Hypothesis

Phase 3-V (rolling-shutter GömbSoma) is a baseline. GömbSoma-Stim beats it, beats the prior HyMeYOLO `+ricci+kcycle` aggregator (0.204 ± 0.04 mAP50 on Cluttered MNIST), and approaches or matches the strong HyMeYOLO baseline `+ricci-mod` (0.723 ± 0.18 mAP50). If GömbSoma-Stim reaches ≥ 0.5 mAP50 on the same task at the same parameter count, the stimulus-driven sensorimotor-stack hypothesis is empirically supported.

3 Architecture

3.1 Pipeline

Stage	Module
1. Image patchify	<code>PatchPatchifier</code> (reuses Phase 3-V <code>patchify</code>)
2. Initial embedding	<code>Linear(patch_dim $\rightarrow$ d_hidden)</code>
3. Saliency scoring	<code>SaliencyHead</code> : MLP $\rightarrow$ scalar per patch
4. Top-K anchor selection	<code>TopKSelector</code> : differentiable in training, hard at inference
5. Hyperedge construction	<code>StimulusGraphBuilder</code> : similarity edges + σ signs
6. GömbSoma layers	Walk / Polygon / Triangle (Phase 1-3-G, unchanged)
7. Detection head	per-anchor class + offset (or per-image classification)

3.2 Saliency scoring

$$s_v = \sigma_{\text{sig}}(w_s^\top \text{GELU}(W_s x_v + b_s)) \in [0, 1]$$

where W_s, b_s, w_s are learnable. Trained jointly with the detection loss; no auxiliary saliency supervision needed — the gradient through top-K selection (Gumbel-softmax in training) pulls s_v up for anchors that contribute to correct detections.

3.3 Top-K selection

Training: Gumbel-softmax over $\{s_v\}$ with temperature τ , producing a soft selection mask $m \in [0, 1]^{|V|}$ that sums to K . Differentiable; gradients flow to the saliency head.

Inference: hard top- K — pick the K largest s_v . Same graph downstream; the soft-vs-hard difference vanishes as $\tau \rightarrow 0$.

3.4 Hyperedge construction among anchors

Given the K selected anchors $A \subset V$, build a signed hypergraph on A with the following primitives:

- *Walks* (w_2, w_3): all length-2 and length-3 walks in the anchor-feature-similarity k -NN graph (each anchor connects to its $\nu = 4$ most-similar other anchors).
- *Polygons* (c_4, c_5, c_6): all small cycles in the same k -NN graph.
- *Triangles* (c_3): all triads of mutually- k NN anchors.
- *Edge signs*: for each edge (u, v) , $\sigma(u, v) = \text{sign}(\langle x_u, x_v \rangle - \theta)$ where θ is a learned threshold (effectively a feature- inner-product polarity).
- *Walk / polygon / triangle signs*: σ -product over constituent edges, as before.

The hypergraph is constructed per-image at training time and is differentiable end-to-end (top-K via Gumbel; k -NN via top- ν on the soft similarity matrix).

3.5 Detection head

Per-anchor classification (YOLO-style):

$$\text{cls}(a) = W_c h_a + b_c, \quad \text{offset}(a) = W_o h_a + b_o$$

where h_a is the anchor’s feature after GömbSoma layers. An anchor with $s_a < \tau_{\text{conf}}$ is treated as background.

Per-image classification (MNIST-only): global pool over anchor features, single softmax. (Simpler; used as warm-up.)

4 Affected files

All under `signedkan_wip/`, no `CORE.YAML` touches:

- `src/hymeko_gomb/soma/vision/stim_anchor.py` — `SaliencyHead`, `TopKSelector`, `StimulusAnchorEnc`

- `src/hymeko_gomb/soma/vision/stim_graph.py` — `StimulusGraphBuilder` (signed-similarity k -NN hypergraph).
- `src/hymeko_gomb/soma/vision/stim_classifier.py` — `StimDrivenImageClassifier` (per-image classification head).
- `src/hymeko_gomb/soma/vision/stim_detector.py` — `StimDrivenDetector` (per-anchor cls + offset).
- `tests/test_gomb_soma_vision_stim_anchor.py`
- `tests/test_gomb_soma_vision_stim_graph.py`
- `tests/test_gomb_soma_vision_stim_classifier.py`
- `tests/test_gomb_soma_vision_stim_detector.py`
- `experiments/run_gomb_soma_stim_cluttered_mnist_2026_05_*.sh`

5 CORE.YAML items touched

None.

6 Phased rollout (one per session, per CLAUDE.md §6.5 #11)

1. `SaliencyHead` + `TopKSelector` (Gumbel-softmax + hard top-K) + unit tests.
2. `StimulusGraphBuilder`: anchors $\rightarrow k$ -NN $\rightarrow$ signed hyperedges; unit tests.
3. `StimDrivenImageClassifier`: end-to-end on MNIST; unit tests; smoke train.
4. `StimDrivenDetector`: per-anchor head on Cluttered MNIST; smoke train.
5. Falsification battery against HyMeYOLO baselines + Phase 3-V baseline.

Each phase produces a working module under unit-test coverage. No phase combines.

7 Test strategy

7.1 Unit tests per module

For each new module:

- permutation-equivariance over anchor set;
- Gumbel temperature $\rightarrow 0$ reproduces hard top-K (in training mode);
- gradient flow through saliency head AND through top-K (the Gumbel trick lets gradient pass);
- parameter count within budget;
- end-to-end overfit-2-samples test.

7.2 Falsification battery

Reference	Metric	GömbSoma-Stim prediction
Phase 3-V (rolling-shutter GömbSoma)	MNIST acc.	match or beat
HyMeYOLO +ricci+kcycle	0.204 mAP50	beat
HyMeYOLO +kcycle (broken)	0.204 mAP50	beat
HyMeYOLO +ricci-mod (positive control)	0.723 mAP50	approach
HyMeYOLO boxes+circles	0.715 mAP50	approach

If GömbSoma-Stim cracks ≥ 0.5 mAP50 on Cluttered MNIST at comparable parameter count, the stimulus-driven hypothesis is empirically supported and we have a real architectural contribution to write up.

8 Performance budget

- Peak RSS: ≤ 14 GB (cgroups MemoryMax 16 GB).
- Per-seed wall on Cluttered MNIST (5000 imgs, 50 epochs): ≤ 25 min on the RTX 2070 SUPER (HyMeYOLO baseline was ~ 13 min).
- GPU memory: ≤ 7 GB.
- Falsification battery (5 seeds, 1 dataset): ≤ 2 h total.

9 Risk anticipation

- *Top-K differentiability.* Gumbel-softmax over K selections may exhibit training instability. Fallback: straight- through estimator with hard top-K + soft gradient via the salience head; or sigmoid + threshold (loses cardinality control).
- *Anchor count too small.* K too small under-represents the scene. Budget $K \in \{8, 16, 32\}$, default 16; choose per-dataset empirically (no Optuna, just a few hand-picked values).
- *σ -balance criterion may not bite.* If anchor-pair signs are random-ish on natural images, σ -balance adds noise rather than signal. Mitigation: the learned threshold θ in $\sigma(u, v)$ should drive $\sigma = +1$ for within-object pairs and $\sigma = -1$ for cross-object pairs. Test by inspecting σ -distribution on planted Cluttered MNIST samples.
- *HyMeYOLO baseline reproducibility.* The +ricci-mod 0.723 mAP50 came from a specific training protocol; GömbSoma-Stim should use the same protocol (5000 images, 50 epochs, same data splits) for a fair comparison.
- *Performance-contract preservation.* The Phase 1-3-G layers already enforce `max_cycles` / top- k caps via the `HypergraphConvConfig`; GömbSoma-Stim must inherit. Verified by the `StimulusGraphBuilder` unit tests that cap polygon / walk counts before handing off.
- *Empty-plan-dir hygiene.* If GömbSoma-Stim is shelved, the plan dir is deleted before commit; no half-finished implementations left behind.

10 Decision tree

- GömbSoma-Stim ≥ 0.5 mAP50 on Cluttered MNIST $\Rightarrow$ stimulus-driven hypothesis confirmed; write up as a paper contribution: “content-driven hypergraph topology for detection.”
- GömbSoma-Stim $\geq$ HyMeYOLO +**kcycle** (0.204) but $< 0.5 \Rightarrow$ partial confirmation; the architecture is doing something but not enough alone. Combine with Phase 4 (TriangleConv + Cartwright–Harary gate) and re-test.
- GömbSoma-Stim $<$ HyMeYOLO +**kcycle** $\Rightarrow$ stimulus-driven approach was wrong; document as negative result; return to Phase 3-V rolling-shutter as the canonical GömbSoma vision encoder.
- GömbSoma-Stim matches HyMeYOLO +**ricci-mod** (0.723) $\Rightarrow$ paper-grade contribution; YOLO-replacement plausible at this parameter scale.

11 Why this could actually beat YOLO at its own game

YOLO’s anchor grid is the bottleneck of small-object / cluttered detection. Every anchor predicts independently; the network has no mechanism to say “these three anchors together form a coherent object.” GömbSoma-Stim natively expresses that: a triangle of anchors with $\pi(c) = +1$ is a balanced clique, i.e., a coherent 3-region object hypothesis. The architecture *is* the relational reasoner; the detection head reads off the consensus.

Cluttered MNIST is the canonical falsification target because the clutter is literally circles that look like digits — to beat the clutter you need to reason *about which combinations of regions actually form a digit*, which is precisely what σ -balanced triangles encode.

12 Bottom line

GömbSoma-Stim replaces the rolling-shutter encoder of Phase 3-V with a content-driven anchor selector + dynamic signed hypergraph builder. The Phase 1-3-G layers (Walk, Polygon, Triangle) are unchanged; GömbSoma-Stim is purely a graph-construction-stage swap. The falsification battery is concrete (Cluttered MNIST against four HyMeYOLO baselines plus the rolling-shutter GömbSoma reference). A confirmed result is a paper-grade contribution: a YOLO-class vision architecture built from signed hypergraph primitives, with the architectural mechanism for relational object reasoning that YOLO lacks.